

Single Login Integration Guide

Implement one login in Angular and use the same authentication context in the Blazor client, so users do not need to sign in twice.

Executive Summary

This guide explains how to make Angular the primary login entry point and transfer JWT authentication to Blazor WebAssembly during navigation.

Business Outcome

- Single sign-in experience
- No duplicate login form for users
- Consistent role-based authorization across clients

Technical Constraint

- Angular and Blazor are different origins
- localStorage is isolated by origin
- Token handoff is required

Architecture

1. User logs in on Angular.
2. Angular stores JWT from API.
3. Angular redirects to Blazor login route with token query parameter.

4. Blazor reads token from URL, stores it locally, and updates auth state.
5. Blazor navigates to protected route as authenticated user.

Implementation Steps

Step 1: Add Blazor base URL in Angular config

emp-client/src/app/core/config/client-config.ts

```
export const API_BASE_URL = "https://localhost:7021/api";  
export const BLAZOR_CLIENT_URL = "https://localhost:7280";
```

This keeps endpoints centralized and avoids hardcoded URLs in components.

Step 2: Build Blazor launch URL in Angular AuthService

emp-client/src/app/services/auth-service.ts

```
import { API_BASE_URL, BLAZOR_CLIENT_URL } from '../core/config/client-config';  
  
getBlazorLaunchUrl() {  
  const token = this.getToken();  
  if (!token) return null;  
  
  // Prevent query string breakage by encoding JWT  
  const encodedToken = encodeURIComponent(token);  
  return `${BLAZOR_CLIENT_URL}/login?token=${encodedToken}`;  
}
```

The token is URL encoded before navigation to preserve all special characters.

Step 3: Add Open Blazor App action in Angular navbar

emp-client/src/app/components/main-nav/main-nav.ts

```
openBlazorApp() {  
  const url = this.authService.getBlazorLaunchUrl();  
  if (!url) {  
    this.router.navigate(["/login"]);  
    return;  
  }  
  
  window.location.href = url;  
}
```

emp-client/src/app/components/main-nav/main-nav.html

```
@if(isLoggedIn()){  
  <li class="nav-item">  
    <button class="btn btn-link nav-link" (click)="openBlazorApp()">  
      Open Blazor App  
    </button>  
  </li>  
}
```

Step 4: Turn Blazor login page into token consumer

EmployeeClient/Pages/Login.razor

```
@page "/login"  
  
@inject CustomAuthProvider AuthServiceProvider  
@inject NavigationManager Nav
```

```
@inject IJSRuntime JS

@if (isProcessingToken)
{
    <p>Signing you in and redirecting...</p>
}
else
{
    <div class="alert alert-info">
        Login is managed by Angular.
        <a href="http://localhost:4200/login">Go to Angular Login</a>
    </div>
}

@code {
    private bool isProcessingToken = true;

    protected override async Task OnInitializedAsync()
    {
        var token = GetTokenFromQuery();
        if (string.IsNullOrEmpty(token))
        {
            isProcessingToken = false;
            return;
        }

        await JS.InvokeVoidAsync("localStorage.setItem", "token", token);
        AuthStateProvider.NotifyUserLoggedIn(token);
        Nav.NavigateTo("/", true);
    }

    private string? GetTokenFromQuery()
    {
        var uri = Nav.ToAbsoluteUri(Nav.Uri);
        var parsed = Microsoft.AspNetCore.WebUtilities.QueryHelpers.ParseQuery(uri.Query);
    }
}
```

```

    if (!parsed.TryGetValue("token", out var values))
    {
        return null;
    }

    var token = values.ToString();
    return string.IsNullOrEmpty(token) ? null : token;
}
}

```

Step 5: Redirect unauthenticated Blazor users to Angular login

EmployeeClient/App.razor

```

<NotAuthorized>
    <p>You are not authorized. Please
        <a href="http://localhost:4200/login">login from Angular</a>.
    </p>
</NotAuthorized>

```

EmployeeClient/Layout/MainLayout.razor

```

<NotAuthorized>
    <li class="nav-item">
        <a class="nav-link" href="http://localhost:4200/login">Login</a>
    </li>
</NotAuthorized>

private async Task HandleLogout()
{
    await JS.InvokeVoidAsync("localStorage.removeItem", "token");
    AuthStateProvider.NotifyUserLoggedOut();
}

```

```
Nav.NavigateTo("http://localhost:4200/login", true);  
}
```

Outcome: one credential prompt in Angular, shared authenticated behavior in both clients.

Security Guidance

Passing raw JWT in URL is acceptable for local development and internal demos only. For production, avoid exposing JWT in query strings.

Production-safe token handoff

1. Angular requests one-time handoff code from API (authenticated call).
2. API stores short-lived mapping code to user context.
3. Angular redirects to Blazor with code parameter, not JWT.
4. Blazor exchanges code for token via API.
5. API invalidates the code after first use.

Testing Checklist

Test	Expected Result
Angular login	JWT stored in Angular localStorage
Navigate to Blazor from Angular	URL contains token parameter in development flow
Blazor login consumer route	Token stored in Blazor localStorage, auth state updated
Role-protected Blazor pages	AuthorizeView and route authorization work as expected

Test	Expected Result
Logout from Blazor	Token cleared and redirected to Angular login

Troubleshooting

- 401 in Blazor: confirm token exists in Blazor localStorage origin.
- Missing roles: verify JWT role claim names align with authorization checks.
- Loop on login route: confirm token is consumed once, then redirect to root.
- CORS errors: allow both Angular and Blazor origins in API CORS policy.

Prepared by: _____ | Reviewed by: _____ | Version: 1.0